Introduction to Computer Science and Programming

Labo 6 – Exercises about Grammars

Laboratory 6, November 11, 2025

Filippo Ricca

DIBRIS, University of Genova

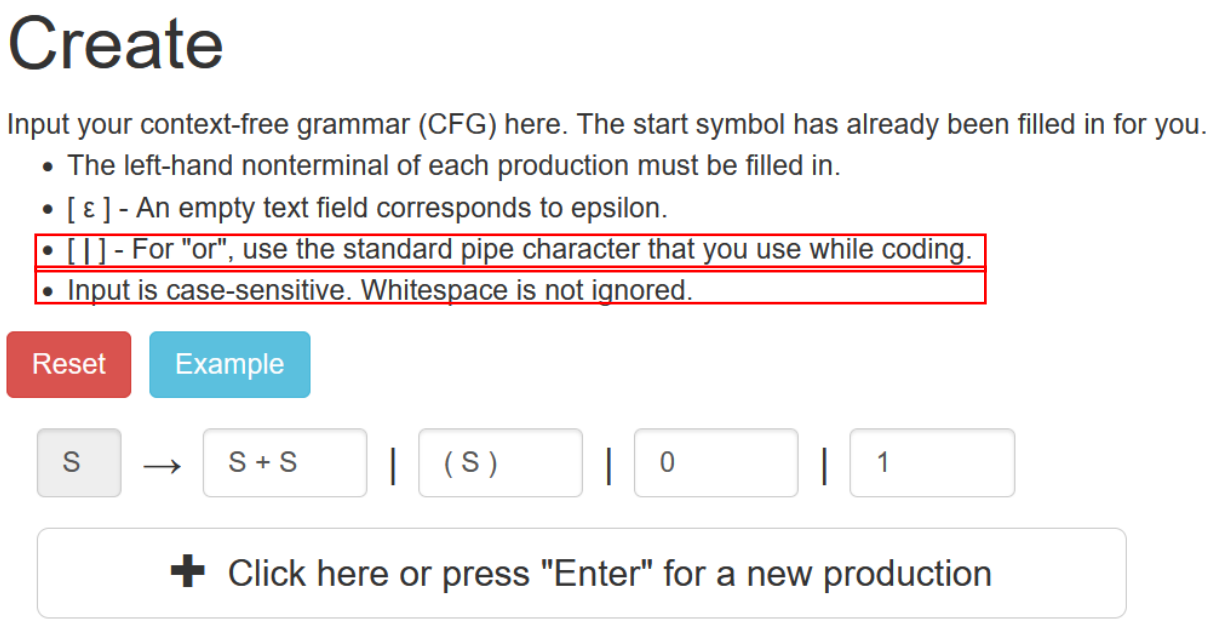
Today…

Today's lab is dedicated to **various exercises on BNF grammars**

- First, we will introduce a **Grammar developer tool**
  - https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/
- **Exercise 1**: Define a BNF Grammar for a simple language
- **Exercise 2**: Parse tree generation
- **Exercise 3**: Grammar derivation
- Exercise to be delivered (not evaluated)
  - Simple programming language

Grammar Developer Tool

- https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/
- **Reproduce our sample Grammar**
- <Sum> ::= 0 | 1 | <Sum> + <Sum> | (<Sum>)



Verify

Verify

This is the CFG you have input above:

Start symbol: **S**

S → **S** + **S** | (**S**) | 0 | 1

Some strings from the language of this grammar:

```
1 + 0 + 0 + 0
0 + 1 + 0 + 1
( 1 )
0
0 + 0
( 1 + 0 )
( 1 + 1 ) + 1
1 + 1 + 0 + 1
( 0 ) + 1
1 + 1
```

Test 1

Test

To test the CFG above, input test strings here, one per line. An empty line corresponds to the empty string. Results will be shown automatically.

```
0+1
```

Test Results for CFG

#	String	Matches	
1	"0+1"	Yes	See Derivation

Rule	Application	Result
Start → S	Start	S
S → S+S	S	S+S
S → 0	S+S	0+S
S → 1	0+S	0+1

Test 2

Test

To test the CFG above, input test strings here, one per line. An empty line corresponds to the empty string. Results will be shown automatically.

```
0 + 1 00
```

Test Results for CFG

#	String	Matches	
1	"0 + 1"	Yes	See Derivation

Rule	Application	Result
Start → S	Start	S
S → S + S	S	S + S
S → 0	S + S	0 + S
S → 1	0 + S	0 + 1

2	"00"	No	

Test 3

Test

To test the CFG above, input test strings here, one per line. An empty line corresponds to the empty string. Results will be shown automatically.

```
0+1+1
```

Test Results for CFG

#	String	Matches	
1	"0+1+1"	Yes (ambiguously)	Derivation One Derivation Two

Rule	Application	Result
Start → S	Start	S
S → S+S	S	S+S
S → S+S	S+S	S+S+S
S → 0	S+S+S	0+S+S
S → 1	0+S+S	0+1+S
S → 1	0+1+S	0+1+1

Submit the code on AulaWeb

Exercise 1: BNF Grammar for our example

- **Objective:** Using the CFG Developer tool, design a **BNF (Backus–Naur Form)** grammar for the language we introduced in the lecture that generates the following strings:

```
1 + 0 + 0 + 0
0 + 1 + 0 + 1
( 1 )
0
0 + 0
```

- The language is generated by the following Grammar:

Start symbol: **S**

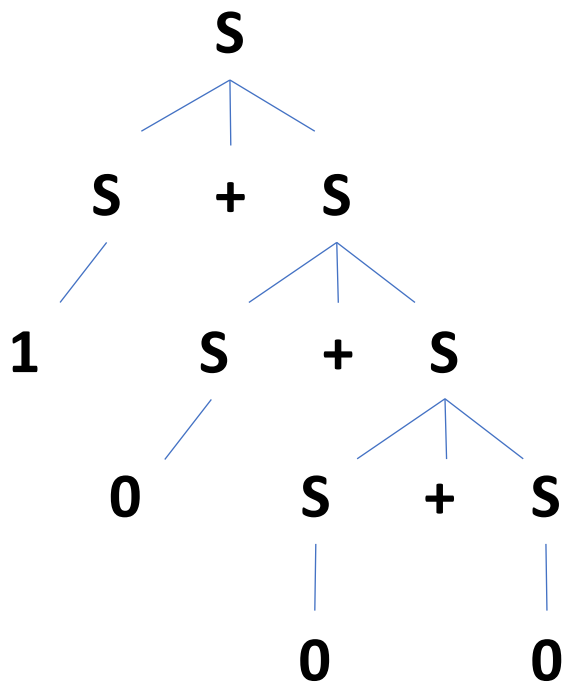
S → **S** + **S** | (**S**) | 0 | 1

Parse Tree Generation

Produce a Parse tree for 1+0+0+0 with the Grammar

Start symbol: **S**

S → **S** + **S** | (**S**) | 0 | 1



Submit the code on AulaWeb

Exercise 2: BNF Grammar for a Simple List Language

- **Objective:** Using the CFG Developer tool, design a **BNF (Backus-Naur Form)** grammar for a language that represents **lists of items**. The language should support:
  - Creating lists with comma-separated items
  - Nesting lists within lists
- **Language Requirements:**
  - **List Creation**: Lists are enclosed in square brackets [], with items separated by commas
    - Example: [apple, banana, cherry]
  - **Nested Lists**: Lists can contain other lists as items
    - Example: [apple, [banana, cherry], date]
  - **Items**: Each item can be either a word (e.g., apple, banana) or a nested list
  - **Word Items**: Words consist only of lowercase alphabetic characters

[a], [ab], [a,b],
[[a]], [a,[b]],
[[a,b]], [a,[b,c],d],
[[[a]]], [[a],[b]],
[abc,[de],f]

Exercise 2: Possible BNF Grammar Solution

```
<list> ::= "[" <items> "]"

<items> ::= <item> | <item> "," <items>

<item> ::= <word> | <list>

<word> ::= <letter> | <letter> <word>

<letter> ::= "a" | "b" | "c" | ... | "z"
```

Theory

Exercise 2: Possible BNF Grammar Solution

```
<list> ::= "[" <items> "]"

<items> ::= <item> | <item> "," <items>

<item> ::= <word> | <list>

<word> ::= <letter> | <letter> <word>

<letter> ::= "a" | "b" | "c" | ... | "z"
```

Theory

Implementation

S	→	[I]														
I	→	O	|	O, I	⊗											
O	→	W	|	S	⊗											
W	→	L	|	LW	⊗											
L	→	a	|	b	|	c	|	d	|	e	|	f	|	g	|
h	|	i	|	l	|	m	|	n	|	o	|	p	|	q	⊗

Exercise: Grammar Derivation

- Given the following BNF Grammar where:
  - **Non-terminals**: S, A, B
  - **Terminals**: a, b
  - **Production rules**:

1. $S \rightarrow AB$
2. $S \rightarrow BA$
3. $A \rightarrow aA$
4. $A \rightarrow a$
5. $B \rightarrow Bb$
6. $B \rightarrow b$

- **Task:**
  - For each of the following strings, determine if it can be derived from the grammar G. Show your reasoning or any intermediate derivations, if possible
  - Strings to Check:
  - ‘aabb’, ‘abab’, ‘aaabbb’, ‘abbb’, ‘baab’, ‘bbaa’, ‘aabbb’

String aabb

$$S \xrightarrow{1} AB \xrightarrow{3} aAB \xrightarrow{4} aaB \xrightarrow{5} aaBb \xrightarrow{6} aabb$$

Conclusion: **aabb can be derived from the grammar**

1. $S \to AB$
2. $S \to BA$
3. $A \to aA$
4. $A \to a$
5. $B \to Bb$
6. $B \to b$

String abab

$S \xrightarrow{1} AB \xrightarrow{3} aAB$

This only gives us strings starting with aa and ending in a sequence of b, but no way to interleave a and b as in abab…..

$S \xrightarrow{2} BA \xrightarrow{5} BbA$

This only gives us strings starting with bb and ending in a sequence of a, but no way to interleave a and b as in abab…..

1. $S \to AB$
2. $S \to BA$
3. $A \to aA$
4. $A \to a$
5. $B \to Bb$
6. $B \to b$

Conclusion: **abab can NOT be derived from the grammar**

Summary of results

Summary of Results:

String	Derivable?
aabb	Yes
abab	No
aaabbb	Yes
abbb	Yes
baab	No
bbaa	Yes
aabbb	Yes

1. $S \to AB$
2. $S \to BA$
3. $A \to aA$
4. $A \to a$
5. $B \to Bb$
6. $B \to b$

Submit the code on AulaWeb

Exercise 3: BNF Grammar for parenthesis language

- **Objective**: Using the CFG Developer tool, design a **BNF (Backus-Naur Form)** grammar for **parenthesis language**, where all strings in the parenthesis language consist of balanced pairs of left and right parentheses

Valid	Invalid
()	())(
(())	(
()()	)(
((()())()())	)))))))))(

Submit the code on AulaWeb

Exercise 4: BNF Grammar for a Simple Programming Language

- **Objective**: Using the CFG Developer tool, design a **BNF (Backus-Naur Form)** grammar for a simple programming language that includes:

- Variable declarations
- Variable assignments
- Conditional statements with if-else
- Basic arithmetic operations

Exercise 4: Language Features

- **Variable Declaration:** A variable can be declared with the keyword var, followed by the variable name (which consists of alphabetic characters) and optionally initialized with an integer value
  - Example: var x = 10;
- **Variable Assignment:** A variable can be assigned a new integer value using the assignment operator =
  - Example: x = 5;
- **Arithmetic Expressions:** The language supports basic arithmetic with +, -, *, and /
  - Example: y = x + 2 * 3;
- **End of Statement: Each statement must end with a semicolon ;**
  - The only exception is conditional statements
- **Conditional Statements:** The language supports if-else statements

```
if (x > 5) {
  x = 10;
} else {
  x = 0;
}
```

Exercise 4: Some examples of generated strings

```
var x = 5;
var y = 10;
```

```
var x = 5;
var y = 10;
x = x + y;
if (x > y) { var z = x - y; }
else { z = y - x; }
```

```
var a = 5;
var b = 10;
var c = 3;
if (a < b) {
    if (b > c) {result = b -
c;}
    else {result = c - b;}
} else {result = a - b;}
```